



COMPUTER
ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	SECTION:	CPE21S1
DATE PERFORMED:	September 8, 2026	DATE SUBMITTED:	September 10, 2026
NAME:	Benj Federic C. De Leon	INSTRUCTOR:	Engr. Jimlord M. Quejado

Activity 7.1 Sorting Algorithms Pt1

1. PROCEDURE OUTPUT

ILO A: Create C++ code for implementation of selection, bubble, insertion and merge sort.

SortingAlgo.h

```
#ifndef SORTINGALGO_H
#define SORTINGALGO_H

#include <utility> // For std::swap

//function for selectionSort to find the index of the smallest element
template <typename T>
int routine_smallest(T arr[], int arrSize, int startIndex) {
    int smallestIndex = startIndex;
    for (int i = startIndex + 1; i < arrSize; i++) {
        if (arr[i] < arr[smallestIndex]) {
            smallestIndex = i;
        }
    }
    return smallestIndex;
}

//Bubble Sort
template <typename T>
void bubbleSort(T arr[], int arrSize){
    for(int i = 0; i < arrSize; i++){
        for(int j = i + 1; j < arrSize; j++){
            if(arr[j] < arr[i]){
```

```
        std::swap(arr[j], arr[i]);
    }
}

//Insertion Sort
template <typename T>
void insertionSort(T arr[], int arrSize){
    int k = 1, j;
    T temp;

    while(k < arrSize){
        temp = arr[k];
        j = k - 1;
        while(j >= 0 && temp <= arr[j]){
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = temp;
        k++;
    }
}

//Selection Sort
template <typename T>
void selectionSort(T arr[], int arrSize){
    int POS, temp;

    for(int i = 0; i < arrSize; i++){
        POS = routine_smallest(arr, arrSize, i); // returns the index
of the smallest
        std::swap(arr[i], arr[POS]);
    }
}
```

```
#endif
```

Create an array of elements with random values. The array must contain 100 elements that are not sorted.

randomArr.cpp

```
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "SortingAlgo.h"

const int maxSize = 100;

//Function Prototype
void displayArr(int arr[]);

int main() {
    int unsortedArr[maxSize];
    srand(time(0));

    for(int i = 0; i < maxSize; i++){
        unsortedArr[i] = rand() % 100;
    }

    displayArr(unsortedArr);

    return 0;
}

void displayArr(int arr[]){
    for (int i = 0; i < maxSize; i++) {
        std::cout << arr[i] << " ";
    }
}
```

```

    }

    std::cout << std::endl;
}

```

Output

```

PS C:\Users\Benj Federic De Leon\Documents\CPE21S1\CPE 010-CPE21S1\HoA7p1> cd "c:\Users\Benj Federic De Leon\Documents\CPE21S1\CPE 010-CPE21S1\HoA7p1\"
if ($?) { g++ randomArr.cpp -o randomArr } ; if ($?) { .\randomArr }
34 55 24 26 21 30 39 69 0 20 83 95 8 0 40 2 5 54 91 49 38 40 38 32 6 32 49 30 90 4 36 90 47 78 38 1 5 34 21 39 71 41 3 4 25 51 60 46 72 51 97 8 2 21 81
4 70 20 94 55 56 50 26 68 15 96 90 44 50 34 55 6 62 15 11 96 34 49 98 88 92 84 66 53 15 54 16 68 20 90 16 86 22 45 34 5 68 46 90 15
PS C:\Users\Benj Federic De Leon\Documents\CPE21S1\CPE 010-CPE21S1\HoA7p1>

```

Code Analysis:

The program creates an array of 100 random numbers and prints them out on the screen. The loop that fills the array runs 100 times, and the loop that prints the array also runs 100 times.

A.1. Bubble Sort Technique

A1bubbleSort.cpp

```

#include <iostream>
#include <cstdlib>
#include <ctime>
#include "SortingAlgo.h"

const int maxSize = 100;

//Function Prototype
void displayArr(int arr[]);

```

```
int main() {
    int unsortedArr[maxSize];
    srand(time(0));

    for(int i = 0; i < maxSize; i++){
        unsortedArr[i] = rand() % 100;
    }
    std::cout << "Original Array:\n";
    displayArr(unsortedArr);

    std::cout << "\nBubble sort: \n";
    bubbleSort(unsortedArr, maxSize);

    displayArr(unsortedArr);

    return 0;
}

void displayArr(int arr[]){
    for (int i = 0; i < maxSize; i++) {
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;
}
```

Output

```
PS C:\Users\Benj Federic De Leon\Documents\CPE21S1\CPE 010-CPE21S1\HoA7p1> cd "c:\Users\Benj Federic De Leon\Documents\CPE21S1\CPE 010-CPE21S1\HoA7p1\"
if ($?) { g++ A1bubbleSort.cpp -o A1bubbleSort } ; if ($?) { .\A1bubbleSort }
Original Array:
34 13 9 69 21 4 10 62 12 21 53 25 88 70 29 82 98 61 54 20 57 69 95 62 85 47 7 52 61 64 96 39 6 66 90 74 92 67 61 18 78 12 13 97 9 62 37 42 10 97 38 52
23 93 83 44 47 48 87 75 25 57 72 44 61 27 58 99 25 73 86 32 72 56 45 91 1 92 31 44 69 18 66 91 80 37 88 14 72 83 27 11 11 69 88 23 90 37 89

Bubble sort:
1 4 6 7 9 9 10 10 11 11 12 12 13 13 14 18 18 20 21 21 23 23 25 25 27 27 29 31 32 34 37 37 37 38 39 42 44 44 44 45 47 47 48 52 52 53 54 56 57 57 58 6
61 61 61 62 62 64 66 66 67 69 69 69 70 72 72 72 73 74 75 78 80 82 83 83 85 86 87 88 88 88 89 90 90 91 91 92 92 93 93 95 96 97 97 98 99
PS C:\Users\Benj Federic De Leon\Documents\CPE21S1\CPE 010-CPE21S1\HoA7p1> |
```

Code Analysis:

This program generates 100 random numbers between 0 and 99 and stores them in an array. It first displays these original numbers to the screen, then calls the **bubbleSort** function to arrange them in order from smallest to largest, and finally prints the newly sorted array.

A.2. Selection Sort Algorithm

A2selectionSort.cpp

```
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "SortingAlgo.h"

const int maxSize = 100;

//Function Prototype
void displayArr(int arr[]);

int main(){
    int unsortedArr[maxSize];
    srand(time(0));

    for(int i = 0; i < maxSize; i++){
```

```

        unsortedArr[i] = rand() % 100;
    }

    std::cout << "\nOriginal Array:\n";
    displayArr(unsortedArr);

    std::cout << "\nSelection Sort: \n";
    selectionSort(unsortedArr, maxSize);
    displayArr(unsortedArr);

    return 0;
}

void displayArr(int arr[]){
    for (int i = 0; i < maxSize; i++) {
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;
}

```

Output

```

PS C:\Users\Benj Federic De Leon\Documents\CPE21S1\CPE 010-CPE21S1\HoA7p1> cd "c:\Users\Benj Federic De Leon\Documents\CPE21S1\CPE 010-CPE21S1\HoA7p1\
if ($?) { g++ A2selectionSort.cpp -o A2selectionSort } ; if ($?) { .\A2selectionSort }

Original Array:
79 77 47 50 86 85 88 72 44 21 51 5 33 12 27 52 95 45 46 56 21 56 20 92 67 12 5 58 93 3 69 89 91 4 19 73 61 29 39 71 74 98 85 37 71 68 91 17 77 44 58 5
2 28 27 12 26 24 86 82 54 59 32 20 42 17 21 65 47 67 77 96 61 48 5 0 76 1 68 11 77 41 43 85 62 33 77 44 98 64 69 18 26 75 45 22 81 23 48

Selection Sort:
0 1 2 3 4 5 5 6 11 12 12 12 17 17 18 19 20 20 21 21 21 22 23 24 26 26 27 27 28 29 32 33 33 37 39 41 42 43 44 44 44 45 45 46 47 47 48 48 50 51 52 54 5
56 57 58 58 59 61 61 62 64 65 67 68 68 69 69 71 71 72 73 74 75 76 77 77 77 77 79 81 82 85 85 85 86 86 88 89 91 91 92 93 95 96 98 98
PS C:\Users\Benj Federic De Leon\Documents\CPE21S1\CPE 010-CPE21S1\HoA7p1>

```

Code Analysis:

This program seeds the random number generator using `srand(time(0))`, fills the array `unsortedArr` with 100 random values using a for loop, calls `displayArr` to print the original list, sorts the elements using

selectionSort, and calls **displayArr** again to print the sorted list.

A.3. Insertion Sort Algorithm

A3insertionSort.cpp

```
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "SortingAlgo.h"

const int maxSize = 100;

//Function Prototype
void displayArr(int arr[]);

int main() {
    int unsortedArr[maxSize];
    srand(time(0));

    for(int i = 0; i < maxSize; i++){
        unsortedArr[i] = rand() % 100;
    }
    std::cout << "\nOriginal Array:\n";
    displayArr(unsortedArr);

    std::cout << "\nInsertion Sort: \n";
    insertionSort(unsortedArr, maxSize);

    displayArr(unsortedArr);

    return 0;
}
```

```
void displayArr(int arr[]){
    for (int i = 0; i < maxSize; i++) {
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;
}
```

Output

```
PS C:\Users\Benj Federic De Leon\Documents\CPE21S1\CPE 010-CPE21S1\HoA7p1> cd "c:\Users\Benj Federic De Leon\Documents\CPE21S1\CPE 010-CPE21S1\HoA7p1" ; if ($?) { g++ A3insertionSort.cpp -o A3insertionSort } ; if ($?) { .\A3insertionSort }
```

Original Array:

```
22 25 67 24 80 55 36 5 46 67 7 94 52 58 8 67 25 58 25 32 32 72 52 66 62 82 93 40 70 1 98 3 13 16 28 56 38 54 76 44 65 53 94 84 94 64
3 99 29 50 95 96 47 99 1 64 88 74 66 11 66 80 92 30 8 8 20 25 12 71 24 46 14 90 74 14 71 2 2 46 57 11 74 93 11 12 70 84 43 78 91 70 7
90 0 9 3 36 62 4
```

Insertion Sort:

```
0 1 1 2 2 3 3 4 5 7 8 8 8 9 11 11 11 12 12 13 14 14 16 20 22 24 24 25 25 25 25 28 29 30 32 32 36 36 38 40 43 44 46 46 46 47 50 52 52
3 53 54 55 56 57 58 58 62 62 64 64 65 66 66 66 67 67 67 70 70 70 71 71 72 74 74 74 74 76 78 80 80 82 84 84 88 90 90 91 92 93 93 94 94
94 95 96 98 99 99
```

```
PS C:\Users\Benj Federic De Leon\Documents\CPE21S1\CPE 010-CPE21S1\HoA7p1>
```

Code Analysis:

This program uses a loop to fill an array with 100 random numbers using `rand() % 100`, calls `displayArr` to print the original numbers, sorts them from smallest to largest by passing the array to **insertionSort**, and calls `displayArr` again to print the sorted numbers.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Solve given data sorting problems using appropriate basic sorting algorithms

Candidate 1 Bo Dalton Capistrano

Candidate 2 Cornelius Raymon Agustín

Candidate 3 Deja Jayla Bañaga

Candidate 4 Lalla Brielle Yabut

Candidate 5 Franklin Relano Castro

List of Candidates

Problem: Generate an array $A[0 \dots 100]$ of unsorted elements, wherein the values in the array are indicative of a vote to a candidate. This means that the values in your array must only range from 1 to 5. Using sorting and searching techniques, develop an algorithm that will count the votes and indicate the winning candidate.

Pseudocode

START

```
candidates = ["", "Bo Dalton Capistrano", "Cornelius Raymon Agustin", "Deja Jayla Banaga", "Lalla Brielle Yabut", "Franklin Relano Castro"]
```

MAIN

```
size = 100
```

```
A = array of size 100
```

```
counts = array of size 6
```

```
call generateVotes(A, size)
```

```
print "Unsorted array: "
```

```
call printArray(A, size)
```

```
call insertionSort(A, size)
```

```
print "Sorted array: "
```

```
call printArray(A, size)
```

```
call countVotes(A, size, counts)
```

```
print "Candidates"
```

```
for i = 1 to 5
```

```
    print "Candidate " + i + ": " + candidates[i]
```

```
end for
```

```
print "Vote Result"
```

```
totalVotes = 0
```

```
for i = 1 to 5
```

```
    print "Candidate " + i + " (" + candidates[i] + "): " + counts[i] + " votes"
```

```
    totalVotes = totalVotes + counts[i]
```

```
end for
```

```
winner = call findWinner(counts)
```

```
print "Winning Candidate"
```

```
print "Candidate " + winner + " - " + candidates[winner] + " wins with " + counts[winner] + " votes!"
```

END MAIN

```
generateVotes(A, n)
```

```
start random numbers
```

```
for i = 0 to n - 1
```

```
A[i] = random number from 1 to 5  
end for  
end
```

```
printArray(A, n)  
for i = 0 to n - 1  
  print A[i] + " "  
  if (i + 1) is divisible by 20  
    go to next line  
  end if  
end for  
go to next line  
end
```

```
countVotes(A, n, counts)  
for i = 1 to 5  
  counts[i] = 0  
end for  
for i = 0 to n - 1  
  counts[A[i]] = counts[A[i]] + 1  
end for  
end
```

```
findWinner(counts)  
winner = 1  
for i = 2 to 5  
  if counts[i] > counts[winner] then  
    winner = i  
  end if  
end for  
return winner  
end
```

iloBsupp.h

```
#ifndef SORTING_ALGORITHMS_H  
#define SORTING_ALGORITHMS_H  
  
template <typename T>  
void insertionSort(T arr[], const int N) {  
    int k = 0, J, temp;  
  
    while (k < N) {
```

```
temp = arr[k];

J = k - 1;

while (J >= 0 && temp <= arr[J]) {

    arr[J + 1] = arr[J];

    J--;

}

arr[J + 1] = temp;

k++;

}

}

#endif
```

iloBsupp.cpp

```
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "iloBsupp.h"

void generateVotes(int A[], int n);

void printArray(int A[], int n);

void countVotes(int A[], int n, int counts[6]);
```

```
int findWinner(int counts[6]);

std::string candidates[6] = {
    "",
    "Bo Dalton Capistrano",
    "Cornelius Raymon Agustin",
    "Deja Jayla Banaga",
    "Lalla Brielle Yabut",
    "Franklin Relano Castro"
};

int main() {

    const int size = 100;

    int A[size];
    int counts[6];

    generateVotes(A, size);
    std::cout << "Unsorted array: " << std::endl;
    printArray(A, size);

    insertionSort(A, size);
    std::cout << "\nSorted array: " << std::endl;
    printArray(A, size);

    countVotes(A, size, counts);

    std::cout << "\nCandidates:" << std::endl;
    for (int i = 1; i <= 5; i++) {
        std::cout << "Candidate " << i << ": " << candidates[i] <<
std::endl;
    }

    std::cout << "\nVote Result" << std::endl;
    int totalVotes = 0;
```

```
for (int i = 1; i <= 5; i++) {
    std::cout << "Candidate " << i << " (" << candidates[i] << "):
"
        << counts[i] << " votes" << std::endl;
    totalVotes += counts[i];
}

int winner = findWinner(counts);
std::cout << "\nWinning Candidate" << std::endl;
std::cout << "Candidate " << winner << " - " << candidates[winner]
    << " wins with " << counts[winner] << " votes!" <<
std::endl;

return 0;
}

void generateVotes(int A[], int n) {
    std::srand((unsigned int)std::time(0));
    for (int i = 0; i < n; i++) {
        A[i] = (std::rand() % 5) + 1;
    }
}

void printArray(int A[], int n) {
    for (int i = 0; i < n; i++) {
        std::cout << A[i] << " ";
        if ((i + 1) % 20 == 0) std::cout << std::endl;
    }
    std::cout << std::endl;
}

void countVotes(int A[], int n, int counts[6]) {
    for (int i = 1; i <= 5; i++) counts[i] = 0;
    for (int i = 0; i < n; i++) {
        counts[A[i]]++;
    }
}
```

```

    }
}

int findWinner(int counts[6]) {
    int winner = 1;
    for (int i = 2; i <= 5; i++) {
        if (counts[i] > counts[winner]) winner = i;
    }
    return winner;
}

```

Output

```

PS C:\Users\Benj Federic De Leon\Documents\CPE21S1\CPE 010-CPE21S1\HoA7p1> cd "c:\Users\Benj Federic De Leon\Documents\CPE21S1\CPE 010-CPE21S1\HoA7p1\" ; if ($?) { g++ iloBsupp.cpp -o iloBsupp } ; if ($?) { .\iloBsupp }
Unsorted array:
5 4 2 5 3 3 4 5 1 2 2 5 1 2 1 4 2 4 3 5
5 2 3 1 1 5 1 4 2 4 2 4 4 5 1 1 1 4 3 5
3 3 2 2 3 4 4 2 2 4 1 5 3 3 1 4 2 3 4 4
1 4 2 5 2 5 4 5 1 5 1 2 5 2 5 5 3 2 5 4
5 4 3 2 4 2 2 2 5 2 3 1 4 3 2 4 2 5 5 2

Sorted array:
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2
2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
2 3 3 3 3 3 3 3 3 3 3 3 3 3 3 4 4 4 4 4
4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 5 5
5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5

Candidates:
Candidate 1: Bo Dalton Capistrano
Candidate 2: Cornelius Raymon Agustin
Candidate 3: Deja Jayla Banaga
Candidate 4: Lalla Brielle Yabut
Candidate 5: Franklin Relano Castro

Vote Result
Candidate 1 (Bo Dalton Capistrano): 15 votes
Candidate 2 (Cornelius Raymon Agustin): 26 votes
Candidate 3 (Deja Jayla Banaga): 15 votes
Candidate 4 (Lalla Brielle Yabut): 22 votes
Candidate 5 (Franklin Relano Castro): 22 votes

Winning Candidate
Candidate 2 - Cornelius Raymon Agustin wins with 26 votes!
PS C:\Users\Benj Federic De Leon\Documents\CPE21S1\CPE 010-CPE21S1\HoA7p1>

```

Time Complexity:

$O(n^2)$, because generating the votes, printing them, and counting them only looks at each vote 1 time $O(n)$. Finding the winner only checks 5 numbers $O(1)$.

Code Analysis:

The program generates 100 random votes for 5 candidates, sorts those votes in order, counts the total votes for each candidate, and determines the winner. Generating, printing, and counting the votes are fast operations because the code looks at each vote only 1 time $O(n)$. Because time complexity is decided by the slowest part of a program, the insertion sort step makes the entire program's time complexity $O(n^2)$.

Conclusion:

Selection sort, bubble sort, and insertion sort are three basic algorithms used to arrange a list of numbers in order. **Selection sort** works by finding the smallest number in the list and moving it to the front one by one. **Bubble sort** compares two side-by-side numbers and swaps them if they are in the wrong order until the whole list is sorted. **Insertion sort** builds a sorted list by taking one number at a time and sliding it into its correct position.

All three algorithms take a similar amount of time $O(n^2)$ to sort large lists because they use nested loops to compare numbers multiple times. Even though they are too slow for huge amounts of data, they are simple to write and help beginners understand how sorting works in programming.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: Gemini

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Explanation of concepts.

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: Was used to gather information about the concepts.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.



**COMPUTER
ENGINEERING**

QUEZON CITY

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.